Orientações Trabalho Prático 1
1º Semestre de 2026

1 Introdução

Os Algoritmos Genéticos (AGs), assim como outros algoritmos evolucionários, são uteis na resolução de problemas de otimização com diferentes propriedades. Os AGs podem ter seus componentes fundamentais implementados de diferentes formas, dependendo inclusive de características do problema em questão.

O objetivo deste Trabalho Prático é aplicar o AG em dois tipos de problemas com características distintas: *i*) combinatorial (Seção 1.1) e *ii*) contínuo (Seção 1.2). Cada um dos problemas será enunciado abaixo juntamente com recomendações sobre as estratégias de representação, seleção, recombinação e mutação que deverão ser empregadas.

1.1 O problema das N-Rainhas

O problema das N-Rainhas consiste em um tabuleiro de xadrez regular ($N \times N$) em que N rainhas devem ser posicionadas de tal forma que nenhuma rainha consiga capturar as outras com apenas um movimento [1], i.e., elas não podem compartilhar a mesma linha, coluna ou diagonal.

Este problema pode ser formulado como um problema de Satisfação de Restrições, onde busca-se por uma solução, i.e., um posicionamento das N rainhas que satisfaça as restrições enunciadas no parágrafo anterior. Para aplicar um AG na resolução deste problema, deve-se definir os seguintes componentes:

- Representação dos indivíduos (soluções candidatas em $\mathcal{G}$);
- Mecanismo de seleção;
- Um ou mais operadores de variação;
- Um critério de parada, e.g., número máximo de avaliações da função aptidão ou um valor de aptidão.

Sugestões de implementação:

- Representação:
  - Fenótipo: Configuração do tabuleiro $N \times N$;
  - Genótipo, ou cromossomo: Permutação dos inteiros $1, 2, 3, \ldots, N$. Por exemplo, a solução apresentada na Figura 1 pode ser representada como $\mathbf{g}_i = [6, 3, 7, 4, 1, 8, 2, 5]$ onde o primeiro valor indica que existe uma rainha na 6ª coluna da 1ª linha do tabuleiro, e o segundo valor indica que existe uma rainha na 3ª coluna da 2ª linha do tabuleiro e assim sucessivamente.

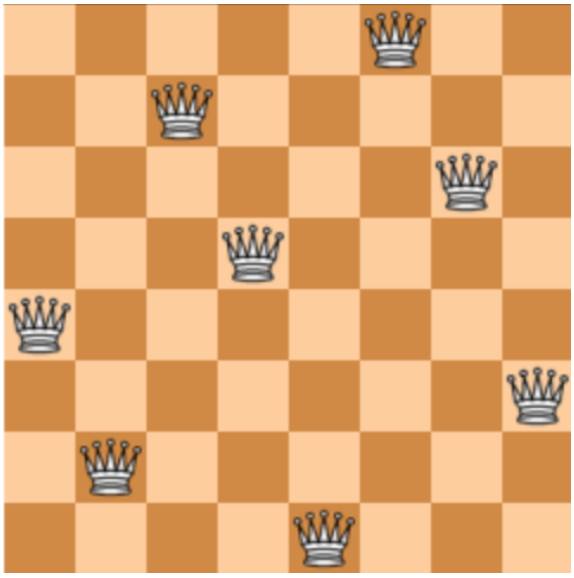

Figura 1: Exemplo de uma solução para o problema de 8-Rainhas.
fonte:By Kestutis Andriejunas - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=82665828

- Operadores de Variação:
  - Recombinação: Gera dois indivíduos filhos (novas soluções candidatas) a partir da combinação de dois indivíduos pais. Os filhos devem ser permutações válidas de N inteiros;
  - Mutação: Seleciona duas posições em um cromossomo aleatoriamente e troca (permuta) os valores destas posições.
- Condição de parada: Encontrar uma solução factível, i.e., $h_j(\mathbf{x}) = 0,\ \forall j$, **ou** quando um número máximo de avaliações da função aptidão tiver ocorrido.

Tarefa: Cada grupo deverá resolver o problema para $N = 8$, i.e., 8 rainhas em um tabuleiro 8×8, com o AG desenvolvido.

Os resultados devem pelo menos relatar o número esperado de gerações para resolver o problema (média de gerações obtida para uma certa quantidade de repetições do experimento). Cada grupo pode modificar ou adicionar outras estratégias de seleção, cruzamento e mutação.

1.2 Função Rastrigin

Existem diferentes funções[1] propostas para a avaliação de desempenho de algoritmos de otimização [1]. Neste trabalho, iremos minimizar uma função não convexa e multimodal conhecida como função Rastrigin. Ela foi proposta por Rastrigin [2] para um domínio bidimensional, conforme ilustrado na Figura 2, e posteriormente generalizada para n dimensões [3].

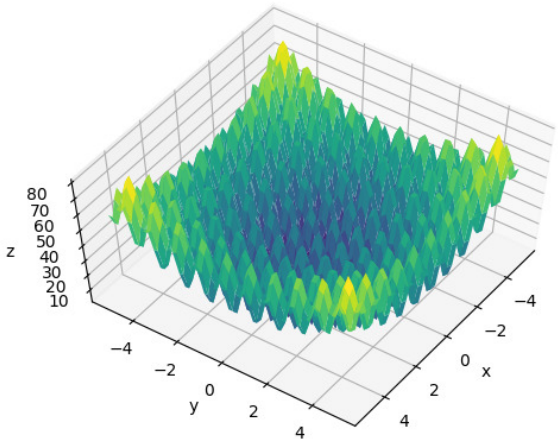

Figura 2: Função Rastrigin bidimensional

Deve-se desenvolver um AG para resolver o problema de minimização da função Rastrigin para $n = 10$ dimensões:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} f(\mathbf{x}) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10\cos(2\pi x_i)\right) \tag{1}$$

com $x_i \in [-5.12,\ 5.12]$. O mínimo global ocorre em $\mathbf{x}^* = 0$ com $f(\mathbf{x}^*) = 0$.
Sugestões de implementação:

- Representação: codificação binária;
- Métodos de seleção:
  - Método da Roleta com 50% de probabilidade;
  - Método do Torneio com 50% de probabilidade.
- Operadores de Variação:
  - Recombinação: 1 ponto de corte por variável;
  - Mutação: bit flip.

[1] Veja https://gitlab.com/luca.baronti/python_benchmark_functions, https://www.sfu.ca/~ssurjano/optimization.html ou https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume24/ortizboyer05a-html/node6.html

Tarefa: Cada grupo deverá resolver o problema para $n = 10$ dimensões. Será necessário definir, ou ajustar dinamicamente, os valores dos hiper-parâmetros p_c e p_m. Todas as implementações devem ter critério de parada de 10^5 avaliações da função objetivo. Cada grupo pode modificar ou adicionar outras estratégias de seleção, cruzamento e mutação.

2 Entrega

- Os trabalhos serão feitos em grupos definidos pelo Professor e disponibilizado no Moodle;
- Deverão ser entregues o **relatório** e os **códigos** implementados até às 23:55 do dia 13/04 pelo Moodle
- Apenas um membro do grupo precisa realizar a entrega;
- Entregas após o prazo terão penalização na nota;
- O texto deve ser entregue em **formato .pdf** e os códigos em um único arquivo **formato .zip** contendo todos os arquivos necessários para sua execução.
- Não serão aceitas entregas em outros formatos.

Os relatórios, em formato IEEE coluna dupla[2], devem introduzir o tema, apresentar a formulação matemática dos problemas, descrever a metodologia de forma reprodutível, apresentar os resultados e uma discussão crítica sobre as estratégias implementadas e uma conclusão. Consultar mais orientações sobre a estrutura no pdf da Aula 0.

O relatório deve conter **pelo menos** as seguintes seções:

- Introdução;
- Formulação do problema;
- Metodologia;
- Resultados e Discussão;
- Conclusão;
- Referências.

Não é necessário fazer uma busca bibliográfica, mas é necessário citar e apresentar as fontes das informações apresentadas.

Referências Bibliográficas

[1] A. E. Eiben and J. E. Smith, *Introduction to evolutionary computing*. Springer, 2015.

[2] L. A. Rastrigin, "Systems of extremal control," in *Theoretical Foundations of Engineering Cybernetics Series*, Moscow: Nauka, 1974.

[3] H. Mühlenbein, M. Schomisch, and J. Born, "The parallel genetic algorithm as function optimizer," *Parallel computing*, vol. 17, no. 6-7, pp. 619–632, 1991.

[2] https://www.ieee.org/conferences/publishing/templates.html ou https://www.overleaf.com/latex/templates?q=ieee